

Firebase

We need these three because they solve **different layers** of Firebase integration in Flutter.

- **Firebase CLI** = manages your Firebase project in the cloud
- **FlutterFire CLI** = connects your Flutter app to that Firebase project
- **firebase_core** = package lets your Flutter app start and use Firebase at runtime

Create firebase account then firebase project in firebase

Install Node js: `winget install OpenJS.NodeJS.LTS`

Set execution Policy: `Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`

Install firebase CLI:

Node: `npm install -g firebase-tools`

Check Version: `firebase --version`

Login: `firebase login`

Create Flutter Project and Install flutterfire_cli:

- Dev dependency: `flutter pub add -d flutterfire_cli`

Install flutterfire

- Dev dependency: `dart run flutterfire_cli:flutterfire configure`

Install firebase_core

- `flutter pub add firebase_core`

Key Features of the Firebase

- **Real-time database:** The firebase is real-time NoSQL database allows the developers to store and sync the data across all clients in real-time.
- **Cloud Firestore:** Firestore is the Firebase's scalable NoSQL cloud database that offers an automatic scaling, high performance, and the real-time capabilities.
- **Firebase Authentication:** Firebase Authentication helps the developers to implement secure authentication mechanisms using by various identity provider such as Google, Facebook, Twitter, GitHub and email/password.
- **Cloud Functions:** Cloud Functions allows the developer to write server-sides code that automatically runs in response to the events triggered by Firebase features and the HTTPs requests.

- **Firebase Cloud Messaging(FCM):** Firebase Cloud Messaging provides the way to send notifications to users across the platforms(iOS,Android and the web).
- **Firebase Hosting:** Firebase Hosting offer the fast, secure and reliable hosting for a static and dynamic content. It is optimized for the web and mobile app hosting, providing SSL certificates by the default for security.

Advantages of Firebase

- **Simplified App Development:** Firebase abstracts much of the complexity that associated with backend services such as authentication, database and the cloud functions which allow developers to focus on creating front-end and the business logic of apps.
- **Real-time Data Synchronization:** With the real-time database and the Firestore, Firebase make it easier to build the apps that require live updates.
- **Cross-Platform Support:** Firebase offer the strong support for cross-platform apps that includes Android, iOS and Web. The unified API across all platforms ensures that the developers do not have to write seperate codebases for each the platform when using Firebase Services.
- **Security:** The Firebase provides built-in security rules and the policies to protect all data accesses and prevent the unauthorized usage.
- **Scalability:** The Firebase automatically scales with our app's growth. Whether we have the small user base or millions of users. Firebase can handle the load without manual intervention.

Use Cases of Firebase

- **Social Media and Chat Apps:** Firebase is the real-time database which is perfectly suited for building the chat apps where messages need to be delivered instantaneously across all the users.
- **Collaborative Platforms:** Applications that allow for multiple users to collaborative in the real-time, such as document editing tools or the project management apps that benefit from Firebase's data synchronization capabilities.
- **E-commerce Apps:** Firebase Authentication and the Cloud Firestore are frequently used in the e-commerce apps for managing the user accounts, product catalogs and the orders.
- **Game Development:** The Firebase's cloud functions, real-time analytics and the performance monitoring make it the popular coice for game developes looking to manage the back-end infrastructure while optimizing the user experience.

Key elements of Firebase architecture include:

- **Backend-as-a-Service(BaaS):** The firebase abstracts back-end complexities and it allows the developers to use the APIs to interact with services.
- **Event-driven architecture:** Firebase uses the triggers(events) to invoke cloud functions or the other services automatically.
- **Data synchronization:** Firebase keeps all the connected clients updated in real-time, making it ideal for the interactive apps.

Store data in database

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:cloud_firestore/cloud_firestore.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: LoginPage(),
    );
  }
}

class LoginPage extends StatefulWidget {
  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  TextEditingController emailController = TextEditingController();

  TextEditingController passwordController =
    TextEditingController();

  Future<void> saveData() async {
```

```

    await FirebaseFirestore.instance.collection("users").add({
      "email": emailController.text,
      "password": passwordController.text,
    });

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("Data Saved")),
    );

    emailController.clear();
    passwordController.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Login Page"),
        centerTitle: true,
      ),
      body: Padding(
        padding: EdgeInsets.all(20),
        child: Column(
          children: [
            TextField(
              controller: emailController,
              decoration: InputDecoration(
                labelText: "Email",
                border: OutlineInputBorder(),
              ),
            ),
            SizedBox(height: 20),

```

```

        TextField(
          controller: passwordController,
          obscureText: true,
          decoration: InputDecoration(
            labelText: "Password",
            border: OutlineInputBorder(),
          ),
        ),
        SizedBox(height: 20),
        ElevatedButton(
          onPressed: saveData,
          child: Text("Login"),
        ),
      ],
    ),
  ),
);
}
}

```

Authentication Login and Register:

```

import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_auth/firebase_auth.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override

```

```

Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: LoginPage(),
  );
}

class LoginPage extends StatefulWidget {
  @override
  _LoginPageState createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  TextEditingController emailController = TextEditingController();
  TextEditingController passwordController =
    TextEditingController();

  Future<void> loginUser() async {
    try {
      await FirebaseAuth.instance.signInWithEmailAndPassword(
        email: emailController.text.trim(),
        password: passwordController.text.trim(),
      );

      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text("Login Successful")),
      );
    } on FirebaseAuthException catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.message.toString())),
      );
    }
  }
}

```

```

}

Future<void> registerUser() async {
  try {
    await FirebaseAuth.instance.createUserWithEmailAndPassword(
      email: emailController.text.trim(),
      password: passwordController.text.trim(),
    );

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text("Account Created")),
    );
  } on FirebaseAuthException catch (e) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(e.message.toString())),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("Firebase Auth"),
      centerTitle: true,
    ),
    body: Padding(
      padding: EdgeInsets.all(20),
      child: Column(
        children: [
          TextField(
            controller: emailController,

```

```

        decoration: InputDecoration(
          labelText: "Email",
          border: OutlineInputBorder(),
        ),
      ),
      SizedBox(height: 20),
      TextField(
        controller: passwordController,
        obscureText: true,
        decoration: InputDecoration(
          labelText: "Password",
          border: OutlineInputBorder(),
        ),
      ),
      SizedBox(height: 20),
      ElevatedButton(
        onPressed: loginUser,
        child: Text("Login"),
      ),
      SizedBox(height: 10),
      ElevatedButton(
        onPressed: registerUser,
        child: Text("Register"),
      ),
    ],
  ),
),
);
}
}

```


Authentication Error Codes:

- **email-already-in-use:**

Thrown if there already exists an account with the given email address.

- **invalid-email:**

Thrown if the email address is not valid.

- **operation-not-allowed:**

Thrown if email/password accounts are not enabled. Enable email/password accounts in the Firebase Console, under the Auth tab.

- **weak-password:**

Thrown if the password is not strong enough.

- **too-many-requests:**

Thrown if the user sent too many requests at the same time, for security the api will not allow too many attempts at the same time, user will have to wait for some time

- **user-token-expired:**

Thrown if the user is no longer authenticated since his refresh token has been expired

- **network-request-failed:**

Thrown if there was a network request error, for example the user doesn't have internet connection

- **operation-not-allowed:**

Thrown if email/password accounts are not enabled. Enable email/password accounts in the Firebase Console, under the Auth tab.